ksformat :: Cheat Sheet

SAS-style **PROC FORMAT** for R — value→label mappings, numeric ranges, reverse formatting (invalue), date/time/datetime, expression labels, multilabel & more

v0.3.4
Vladimir Larchenko · GPL-3
`install.packages("ksformat")`

INSTALL & LOAD

```
# From CRAN
install.packages("ksformat")
# From GitHub (dev)
remotes::install_github("crow16384/ksformat")
library(ksformat)
```

FORMAT CREATION

FNEW() — DISCRETE VALUE→LABEL VALUE

```
fnew( ..., name=NULL, type="auto", default=NULL, multilabel=FALSE, ignore_case=FALSE)
  ... = named pairs key="label" | .missing/.other = special labels | name = register in library
```

```
fnew(
  "M" = "Male",
  "F" = "Female",
  .missing = "Unknown",
  .other = "Other Gender",
  name = "sex"
)
```

sex CHAR
- M → Male
- F → Female
- .missing → Unknown
- .other → Other Gender

FINPUT() — REVERSE (LABEL→VALUE) INVALUE

```
finput( ..., name=NULL, target_type="numeric", missing_value=NA)
  ... = named pairs label=value | target_type = "numeric"|"character" output type
```

```
finput(
  "Male" = 1,
  "Female" = 2,
  name = "sex_inv"
)
```

sex_inv INVALUE →NUM
- Male → 1
- Female → 2

FNEW_BID() — BIDIRECTIONAL VALUE + INVALUE

```
fnew_bid( ..., name=NULL, type="auto")
  Creates both a VALUE format and matching _inv INVALUE simultaneously
```

```
fnew_bid(
  "A" = "Active",
  "I" = "Inactive",
  name = "status"
)
```

Creates **both**:
- "status" VALUE format
- "status_inv" INVALUE

FNEW_DATE() — DATE/TIME/DATETIME FORMAT

```
fnew_date(pattern, name=NULL, type="auto", .missing=NULL)
  pattern = SAS format name (DATE9.) or R strftime pattern (%d.%m.%Y)
```

```
fnew_date("DATE9.", name = "bday")

fnew_date("%d.%m.%Y",
  name = "ru", type = "date")

fnew_date("MMDDYY10.",
  name = "us",
  .missing = "NO DATE")
```

bday DATE
Pattern: %d%b%Y (DATE9.)

SAS names auto-resolved; R patterns also supported

OPTIONS FOR FNEW()

PARAM	DEFAULT	WHAT IT DOES
name	NULL	Register in format library
type	"auto"	"numeric"/"character" key type
multilabel	FALSE	Allow multiple labels per value
ignore_case	FALSE	Case-insensitive matching
.missing	–	Label for NA/NaN/""
.other	–	Fallback for unmatched values

FORMAT APPLICATION

FPUT() — APPLY FORMAT (GENERIC)

```
fput(x, format, ... , keep_na=FALSE)
  x = values | format = name or object | ... = extra args for expression labels
```

```
fput(
  c("M", "F", "M", NA, "X"),
  "sex"
)
```

"Male" "Female" "Male" "Unknown"
"Other Gender"

FPUTN() FPUTC() — TYPED APPLY

```
fputn(x, format_name, ...) · fputc(x, format_name, ...)
  Numeric/character shorthand — format_name is always a string name
```

```
fputn(c(5, 30, 70), "age")

fputc(c("M", "F"), "sex")
```

"Child" "Adult" "Senior"
"Male" "Female"

FPUT_DF() — FORMAT DATA FRAME COLUMNS

```
fput_df(data, ... , suffix="_fmt", replace=FALSE)
  ... = col=format pairs | suffix = new column suffix | replace = overwrite original
```

```
fput_df(df,
  sex = format_get("sex"),
  age = format_get("age"),
  suffix = "_lbl"
)
```

SEX	AGE	SEX_LBL	AGE_LBL
M	15	Male	Child
F	25	Female	Adult
NA	35	Unknown	Adult

CASE-INSENSITIVE MATCHING

```
fnew(
  "M" = "Male",
  "F" = "Female",
  name = "sex_nc",
  ignore_case = TRUE
)
fput(c("m", "F", "M"), "sex_nc")
```

"Male" "Female" "Male"
"m" matches "M" with ignore_case

MISSING VALUE HANDLING

#	CONDITION	RESULT
①	NA / NaN / ""	.missing label
②	Exact match	Mapped label
③	Range match	Range label
④	No match	.other / original

IS_MISSING() — CHECK FOR MISSING VALUES

```
is_missing(x)
  Returns TRUE for NA, NaN, and empty string ""
```

```
is_missing(NA)    # TRUE
is_missing(NaN)   # TRUE
is_missing("")    # TRUE
is_missing("x")   # FALSE
```

TRUE
TRUE
TRUE
FALSE

KEEP_NA OPTION

```
fput(c("M", NA), "sex")

fput(c("M", NA), "sex",
  keep_na = TRUE)
```

"Male" "Unknown"
"Male" NA

TEXT PARSING & NUMERIC RANGES

FPARSE() — PARSE VALUE/INVALUE TEXT

```
fparse(text=NULL, file=NULL)
  text = format definition string | file = path to .txt file with definitions
```

```
fparse(text = '
VALUE age (numeric)
  [0, 18)    = "Child"
  [18, 65)   = "Adult"
  [65, HIGH] = "Senior"
  .missing   = "Age Unknown"
;')
fputn(c(5,18,65,NA), "age")
```

"Child" "Adult" "Senior" "Age Unknown"

RANGE SYNTAX REFERENCE

SYNTAX	MEANING	MATH
[a, b]	both inclusive	a ≤ x ≤ b
[a, b)	right exclusive	a ≤ x < b
(a, b]	left exclusive	a < x ≤ b
(a, b)	both exclusive	a < x < b
HIGH	+∞ upper bound	
LOW	−∞ lower bound	

BMI WITH DECIMALS + .MISSING

```
fparse(text = '
VALUE bmi (numeric)
  [0, 18.5)  = "Underweight"
  [18.5, 25) = "Normal"
  [25, 30)   = "Overweight"
  [30, HIGH] = "Obese"
  .missing   = "No data"
;')
```

BMI	RESULT
16.2	Underweight
22.7	Normal
25.0	Overweight
35.1	Obese
NA	No data

EXCLUSIVE / INCLUSIVE BOUNDS + .OTHER

```
fparse(text = '
VALUE score (numeric)
  (0, 50]   = "Low"
  (50, 100] = "High"
  .other    = "Out of range"
;')
```

SCORE	LABEL
0	Out of range
50	Low
51	High
101	Out of range

PARSE MULTIPLE FORMATS AT once

```
fparse(text = '
VALUE race (character)
  "W"="White" "B"="Black"
  "A"="Asian"
  .missing = "Unknown"
;
INVALUE race_inv
  "White"=1 "Black"=2
  "Asian"=3
;')
```

Registers **both** in library:
- race VALUE (char)
- race_inv INVALUE (num)

RANGE_SPEC() — BUILD RANGE STRINGS

```
range_spec(low, high, label, inc_low=TRUE, inc_high=FALSE)
  Programmatic range key builder for fnew() — returns "low,high,inc_low,inc_high"
```

```
range_spec(0, 18)

range_spec(18, Inf,
  inc_high = FALSE)
```

"0,18,TRUE,FALSE"
"18,Inf,TRUE,FALSE"

REVERSE FORMATTING (INVALUE)

FINPUTN() — LABELS → NUMERIC

```
finputn(x, invalue_name)
  Applies named invalue, returns numeric vector
```

```
finputn(
  c("Male", "Female", "Unknown"),
  "sex_inv"
)
```

1 2 NA

FINPUTC() — LABELS → CHARACTER

```
finputc(x, invalue_name)
  Applies named invalue, returns character vector
```

```
finputc(
  c("Active", "Pending"),
  "status_inv"
)
```

"A" "P"

ROUND-TRIP: VALUE ↔ INVALUE

"A" → fputc "status" → "Active" → finputc "status_inv" → "A"

DATE, TIME & DATETIME

SAS DATE FORMATS (AUTO-RESOLVED)

```
fputn(Sys.Date(), "DATE9.")
fputn(Sys.Date(), "MMDDYY10.")
fputn(Sys.Date(), "YYMMDD10.")
fputn(Sys.Date(), "WORDDATE.")
fputn(Sys.Date(), "QTR.")
```

FORMAT	→ RESULT
DATE9.	18MAR2026
MMDDYY10.	03/18/2026
YYMMDD10.	2026-03-18
DDMMYY10.	18/03/2026
MONYY7.	MAR2026
WORDDATE.	March 18, 2026
YEAR4.	2026
QTR.	1

R NUMERIC DATES (DAYS SINCE 1970-01-01)

```
days <- as.numeric(
  as.Date("2025-01-01")
)
fputn(days, "DATE9.")
fputn(days, "MMDDYY10.")
```

days = 20089
"01JAN2025"
"01/01/2025"

TIME FORMATS (SECONDS SINCE MIDNIGHT)

SECS	TIME8.	TIME5.	HHMM.
0	0:00:00	0:00	00:00
3600	1:00:00	1:00	01:00
45000	12:30:00	12:30	12:30
86399	23:59:59	23:59	23:59

DATETIME FORMATS

```
fputn(Sys.time(),
  "DATETIME20.")
```

FORMAT	→ RESULT
DATETIME20.	18MAR2026:13:48:58
DATETIME13.	18MAR26:13:48
DTDATE.	18MAR2026
DTYYMMDD.	2026-03-18

CUSTOM + DATA FRAMES

```
fnew_date("%d.%m.%Y",
  name = "ru_date")

fput(birthdays, "ru_date")
```

"25.03.1990" "03.11.1985" "14.07.2000"

```
v <- fnew_date("DATE9.",
  name = "v_fmt",
  .missing = "NOT RECORDED")

fput_df(patients,
  visit_date = v)
```

ID	VISIT_DATE	VISIT_FMT
1	2025-01-10	10JAN2025
2	2025-02-15	15FEB2025
3	<NA>	NOT RECORDED

PUTN WORKFLOW (SAS-STYLE DYNAMIC DISPATCH)

```
fnew(
  "1" = "date9.",
  "2" = "mmddyy10.",
  name = "w_fmt",
  type = "numeric")

datefmt <- fputn(key, "w_fmt")
date <- fputn(number, datefmt)
```

NUM	KEY	FMT	DATE
12103	1	date9.	03FEB2003
10899	2	mmddyy10.	11/07/1999

PARSE DATE FORMATS FROM TEXT

```
fparse(text = '
VALUE enrldt (date)
  pattern = "DATE9."
  .missing = "Not Enrolled"
;
VALUE stamp (datetime)
  pattern = "DATETIME20."
;')
```

FORMAT LIBRARY

FPRINT() — INSPECT REGISTERED FORMATS

```
fprint(name=NULL)
  No args = list all | name = show detail for one format
```

```
fprint()       # list all
fprint("sex")  # detail
```

- age VALUE (num) 3
- sex VALUE (char) 2
- sex_inv INVALUE (num) 2

FORMAT_GET() FCLEAR()

```
format_get(name) — retrieve format object    fclear(name=NULL) — remove one or all
```

```
fmt <- format_get("sex")

fclear("sex")  # remove one
fclear()       # clear all
```

Retrieve format object
Remove single format
Clear entire library

FEXPORT() — EXPORT TO PARSEABLE TEXT

```
fexport( ... , formats=NULL, file=NULL)
  ... or formats = format objects | file = write to file (else returns string)
```

```
cat(fexport(bmi = bmi_fmt))
```

```
VALUE bmi (numeric)
  [0, 18.5) = "Underweight"
  [18.5, 25) = "Normal"
  [25, 30) = "Overweight"
  [30, HIGH] = "Obese"
;
```

FIMPORT() — SAS CNTLOUT CSV

```
fimport(file, register=TRUE, overwrite=TRUE)
  file = CNTLOUT CSV path | register = auto-add to library | overwrite = replace existing
```

```
x <- fimport("cntlout.csv")

m <- fimport(csv,
  register = FALSE)

fput(v, m[["GENDER"]])
```

- AGEGRP VALUE (num)
- BMICAT VALUE (num)
- GENDER VALUE (char)

MULTILABEL FORMATS

FPUT_ALL() — ONE VALUE → MULTIPLE LABELS MULTILABEL=TRUE

```
fput_all(x, format, ... , keep_na=FALSE)
  Returns list of character vectors — all matching labels per value
```

```
fnew(
  "0,5,T,T"    = "Infant",
  "6,11,T,T"   = "Child",
  "12,17,T,T"  = "Adolescent",
  "0,17,T,T"   = "Pediatric",
  "18,64,T,T"  = "Adult",
  "65,Inf,T,T" = "Elderly",
  "18,Inf,T,T" = "Non-Pediatric",
  name = "age_cat",
  type = "numeric",
  multilabel = TRUE
)

fput_all(
  c(3, 14, 25, 70),
  "age_cat"
)
```

- 3 → Infant, Pediatric
- 14 → Adolescent, Pediatric
- 25 → Adult, Non-Pediatric
- 70 → Elderly, Non-Pediatric

fput() → first match only · fput_all() → all matching labels (list)

AE SEVERITY GRADING (CLINICAL)

```
fnew(
  "1,1,T,T" = "Mild",
  "2,2,T,T" = "Moderate",
  "3,3,T,T" = "Severe",
  "4,4,T,T" = "Life-threat.",
  "5,5,T,T" = "Fatal",
  "3,5,T,T" = "Serious",
  "1,2,T,T" = "Non-serious",
  name = "ae",
  type = "numeric",
  multilabel = TRUE
)
```

- 1 → Mild, Non-serious
- 2 → Moderate, Non-serious
- 3 → Severe, Serious
- 4 → Life-threat., Serious
- 5 → Fatal, Serious

EXPRESSION LABELS

SPRINTF WITH .X1 (EVALUATED AT APPLY-TIME)

```
stat <- fnew(
  "n"   = "sprintf('%s', .x1)",
  "pct" = "sprintf('%.1f%%',
             .x1 * 100)",
  name = "stat"
)

fput(
  c("n", "pct", "n", "pct"),
  stat,
  c(42, .053, 100, .255)
)
```

"42" "5.3%" "100" "25.5%"

TWO ARGS (.X1, .X2) + SCALAR RECYCLING

```
r <- fnew(
  "ratio" = "sprintf('%s/%s',
             .x1, .x2)"
)
fput("ratio", r, 3, 10)

lbl <- fnew(
  "val" = "sprintf('%s(N=%s)',
             .x1, .x2)"
)
fput(c("val", "val"), lbl,
  c(42, 55), 100)
```

"3/10"
"42(N=100)" "55(N=100)"
.x2=100 recycled for all elements

IFELSE + MIXED STATIC/EXPRESSION

```
sign <- fnew(
  "val" = "ifelse(.x1 > 0,
    paste0('+', .x1),
    as.character(.x1))"
)
fput(rep("val", 3), sign,
  c(5, 0, -3))

# Mixed: static + expression
fnew(
  "ok" = "OK",
  .other = "sprintf('Err(%s)',
             .x1)"
)
```

"+5" "0" "-3"
"OK" "Err(timeout)"
"ok" → static · "E01" → .other expression

VECTORIZED FORMAT NAMES

EACH ELEMENT USES A DIFFERENT FORMAT (SAS PUTC/PUTN)

```
fnew(
  "1" = "groupx",
  "2" = "groupy",
  "3" = "groupz",
  name = "typefmt",
  type = "numeric"
)
# define groupx, y, z ...
respfmt <- fput(type,
  "typefmt")
word <- fputc(response,
  respfmt)
```

TYPE	RESP	FMT	WORD
1	positive	groupx	agree
1	negative	groupx	disagree
2	positive	groupy	accept
2	negative	groupy	reject
3	positive	groupz	pass
3	negative	groupz	fail

FUNCTION REFERENCE

CATEGORY	FUNCTION	PURPOSE
Create	fnew(... , name)	Discrete value→label
	finput(... , name)	Reverse label→value
	fnew_bid(... , name)	Both format + invalue
	fnew_date(pattern, name)	Date/time/datetime
Apply	fput(x, format, ...)	Generic apply
	fputn(x, name, ...)	Numeric apply
	fputc(x, name, ...)	Character apply
	fput_all(x, format)	All labels (multilabel)
	fput_df(data, ...)	Data frame columns
Reverse	finputn(x, name)	Labels → numeric
	finputc(x, name)	Labels → character
Library	format_get(name)	Get from library
	fprint(name)	List/inspect
	fclear(name)	Remove format(s)
	fimport(file)	SAS CNTLOUT CSV
Utils	fparse(text, file)	Parse definitions
	fexport(... , file)	Export to text
	range_spec(lo, hi)	Build range key
Check	is_missing(x)	NA/NaN/"" → TRUE